

Stage 2 Report:

Predicting Helsinki City Bike Trip Duration from Route, Time and Weather

CS-C3240 Machine Learning

1. Introduction

Each year, hundreds of thousands of people rent shared city bikes across Helsinki and Espoo. Before setting off, a rider has no simple way to estimate how long the trip to their destination will take — information that could help them decide whether cycling is a reasonable choice given the time and the weather. This project uses machine learning to predict the duration of a city bike trip before it starts. Section 2 formulates the problem and describes the data. Section 3 explains the preprocessing, feature selection, models, loss functions and data split. Section 4 compares the training, validation and test errors of the methods, and Section 5 discusses the results and their limitations.

2. Problem Formulation

We treat the application as a supervised regression problem. Each data point represents one completed trip in the Helsinki/Espoo city bike system. The label is the trip's duration in minutes, a continuous, strictly positive quantity taken from the trip record. Because durations are heavily right-skewed, our models predict its logarithm and convert predictions back to minutes. Each trip is described by the seven features in Table 1.

Table 1. Features describing each trip.

Feature	Type	Description
Route length	Continuous	Typical distance (m) between the start and end station, learned from earlier trips
Round trip	Binary	Whether the bike is returned to the station it was taken from
Weekday	Categorical	Day the trip started (Mon-Sun)
Hour of day	Categorical	Hour the trip started (0-23)
Month	Categorical	Month of the operating season (Apr-Oct)
Mean temperature	Continuous	Daily mean air temperature (°C)
Precipitation	Continuous	Daily total precipitation (mm)

The problem is the same as in Stage 1, but the feature set has changed. Stage 1 excluded distance so that the model could not "cheat". That argument conflated two quantities: the distance *recorded* for a trip is only known after the ride and remains excluded, whereas the *route* — the pair of stations — is known before departure, and our problem statement assumes the rider knows their destination. Stage 2 therefore adds the typical length of the route, computed only from other, earlier trips (Section 3.2).

The trip data comes from HSL's (Helsinki Region Transport) open Origin-Destination dataset [1]. Weather data comes from the Finnish Meteorological Institute's open data for the Kaisaniemi station in Helsinki [2], and is joined to each trip by date. Both are licensed CC BY 4.0.

3. Methods

3.1 Dataset and preprocessing

The dataset covers every trip of the April-October seasons 2022-2025: 28 monthly files with 10,121,026 trips, each recording the departure time, start and end station, distance and duration. We removed rows with missing values, 274,378 "false starts" shorter than 10 seconds or 10 metres (a bike undocked and immediately re-docked), and 1,264 movements to HSL's workshop and test stations. This leaves **9,837,352 data points** (97.2%), covering 463 stations and 110,389 distinct routes. We keep very long trips, which are mostly rentals that were never ended (the longest lasts over 4,600 hours), because removing them would need an arbitrary cut-off; Section 3.4 handles them through the loss instead.

3.2 Feature selection

We selected features using domain knowledge and visualisation. We assumed that duration depends on three factors: the trip's length, the time (through congestion or the kind of trip) and the weather (bad weather slowing riders down). Since duration is distance divided by speed, we plotted median speed and median distance against each candidate factor, using the recorded distance for this exploration only. Length dominated: log distance varies almost six times as much as log speed (variance 0.536 vs 0.093). Time acted through the purpose of the trip rather than congestion: on weekdays, the morning rush is the fastest period of the day (12.6 km/h, against 11.6 km/h at midday). Rain did not slow riders: on days with over 10 mm of rain, median speed was 1.6% higher and distance 7.7% lower than on dry days, which corrects an assumption in our Stage 1 report. We therefore kept the five Stage 1 calendar and weather features, so that their effects could be tested, and added two route features.

Route length is the median recorded distance of other trips between the same two stations — information a journey planner has in advance. So that no trip contributes to its own value, training trips are cross-fitted (2022 trips use 2023 data and vice versa); validation trips use 2022-2023, and test trips 2022-2024. Routes with fewer than three earlier trips fall back to the departure station's median. The **round-trip** flag marks the 3.2% of trips that return to their starting station, which are mostly leisure loops.

3.3 Models (hypothesis spaces)

We compare two hypothesis spaces [3]. The first is **linear predictors**: linear regression on $\log(\text{duration})$, with one-hot encoded weekday, hour and month and standardised numeric features. Because both the label and route length are logged, the model expresses duration as a power of route length, scaled by percentage effects of time and weather. This is a direct form of our "distance divided by speed" hypothesis, which the visualisation supported. The second is **gradient-boosted regression trees** [5], a non-linear hypothesis space that can capture interactions the linear model cannot, such as rain mattering more on long routes. Two reference predictors without learning anchor the results: the training median, and a physics baseline that multiplies route length by one typical pace (5.23 min/km, i.e. 11.5 km/h).

3.4 Loss functions

Trip durations contain extreme outliers: trips over five hours are 0.29% of the data but contribute 99.6% of the squared error around the mean, so a model trained with squared error would mostly fit forgotten rentals. We counter this in two complementary ways. First, we fit $\log(\text{duration})$, which reduces the share of these trips in the squared error to 9.0%. Second, we train the linear model with both the **squared error** and the **Huber loss** [4] ($\epsilon = 1.35$). The Huber loss is quadratic for small residuals and linear for large ones, so it bounds the influence of the outliers that remain; fitting both losses on the same hypothesis space isolates the effect of the loss. The trees use the **absolute error**, a robust loss available in the implementation we used, which offers no Huber option. Our three ML methods are therefore linear regression with squared error, linear regression with the Huber loss, and boosted trees with absolute error.

To evaluate predictions we use, in minutes, the mean absolute error (MAE), the median absolute error (MedAE, the error on a typical trip) and the share of trips predicted within two minutes, the most relevant measure for a rider. We do not evaluate with the squared error, which mainly measures forgotten rentals.

3.5 Training, validation and test sets

We split by season rather than at random, because the model must generalise to future seasons: a random split would put trips from the same day, which share weather and events, in both sets and give an optimistic estimate. The **training set** is the 2022-2023 seasons (4,876,755 trips). All methods are fitted on the same random sample of 400,000 of them, because the Huber solver is slow on millions of rows. The **validation set** is the 2024 season (2,508,741 trips) and the **test set** the 2025 season (2,451,856 trips). Before seeing any results, we fixed the selection rule: the method with the lowest validation MAE is chosen, then retrained on 2022-2024 and evaluated once on the test set. To judge whether differences are real, we compute 95% confidence intervals for paired differences with a bootstrap that resamples whole days [6], since trips on the same day are not independent.

4. Results

4.1 Training and validation errors, and choice of method

Table 2. Errors in minutes, and share of trips predicted within 2 minutes. The chosen method is in bold.

Predictor	Training (sample)		Validation (2024)			Test (2025)		
	MAE	MedAE	MAE	MedAE	≤ 2 min	MAE	MedAE	≤ 2 min
Constant (median)	10.404	4.883	13.978	5.300	19.0%	14.892	5.433	18.6%
Physics (no learning)	7.921	1.755	11.219	2.059	49.1%	11.949	2.140	48.0%
Linear, squared error	7.736	1.983	11.002	2.205	46.1%	11.704	2.353	43.3%
Linear, Huber loss	7.622	1.617	10.909	1.857	52.5%	11.548	1.911	51.5%
Trees, absolute error	7.570	1.576	10.955	1.853	52.5%	11.599	1.890	51.8%

Route length does most of the work: the physics baseline alone lowers the validation MedAE from 5.30 to 2.06 minutes. Training errors are below validation errors for every predictor, including the constant and the physics baseline, which fit at most one parameter. The gap therefore reflects differences between seasons rather than overfitting. The boosted trees reach the lowest training error but have no validation advantage, a mild sign that their extra flexibility fits the training data rather than the problem.

The loss matters even on the log scale. The Huber loss gives a 0.093 min lower validation MAE than squared error (95% CI [0.088, 0.098]) and predicts 6.4 percentage points more trips within two minutes ([6.1, 6.6]). With squared error, the remaining tail pulls predictions upwards by about half a minute, which makes it worse than the physics baseline on typical trips. Between the hypothesis spaces, the linear model's MAE is 0.046 min lower ([0.043, 0.049]) — under three seconds, detectable only because of the 2.5 million validation trips — and the share within two minutes does not differ ([-0.07, 0.06] pp). By the pre-declared rule, we choose **linear regression with the Huber loss**: it has the lowest validation MAE, matches the trees on the other metrics, and is simpler and directly interpretable.

4.2 What drives the predictions

Refitting the chosen model on subsets of features tests our three-factor assumption. On validation MedAE, the Stage 1 features alone reach 5.25 min, barely better than the constant (5.30). Route length and the round-trip flag alone reach 1.865 min, 99.8% of the full model's improvement. Adding time improves this to 1.850 min, while adding weather does not help (1.875 with length; 1.857 with all features). The fitted coefficients agree with the visualisation: duration grows as route length to the power 0.85, trips are fastest in the early morning (−10% at 05:00, −5.5% at 08:00) and slowest in the afternoon (+4.3% at 13:00), weekends are about 3% slower, and rain shortens trips slightly (−0.12% per mm). Finally, removing the longest 0.1% of training trips moves the squared-error model's predictions 12 times more than the Huber model's, relative to removing the same number of random trips (0.46 vs 0.04 percentage points). This confirms that the Huber loss protects against the remaining outliers.

4.3 Test error

The test set, the 2025 season, was used neither to train the methods nor to choose between them. On it, the chosen method has a **test MAE of 11.55 minutes and a test MedAE of 1.91 minutes**, and predicts 51.5% of trips within two minutes (Table 2). This is close to its validation performance (1.86 min, 52.5%), so the model generalises to a new season, and it reduces the constant's typical error by 65%. The ranking of the methods is unchanged: the Huber loss beats squared error by 8.2 percentage points within two minutes (CI [7.9, 8.5]), and the trees and the linear model remain practically tied. The predictions are unbiased: the median ratio of actual to predicted duration is 1.006 overall, and between 0.98 and 1.01 at every hour of the day.

5. Conclusions

We predicted the duration of city bike trips from the route, the time and the weather. The chosen method, linear regression on log(duration) with the Huber loss, predicts trips in an unseen season with a typical error of 1.9 minutes and places half of them within two minutes; a constant guess achieves 5.4 minutes and 19%. The results support three conclusions. The route explains almost all of the predictable variation. Time has a small effect that reflects the purpose of the trip rather than congestion, while daily weather adds nothing.

And the choice of loss matters more than the hypothesis space: the Huber loss clearly outperforms squared error, while boosted trees add nothing over the linear model.

For a typical trip the problem is solved satisfactorily, but there is room for improvement. The MAE (11.5 min) stays far above the MedAE (1.9 min), because forgotten rentals and riders' stops cannot be predicted from these features. The physics baseline already achieves most of the accuracy, so learning adds a real but modest refinement. Daily weather is coarse, route lengths fall back to cruder estimates for 2.6% of test trips (new stations and rare routes), and the models were trained on a 400,000-trip sample. Promising directions are hourly weather observations, station coordinates to estimate the length of unseen routes, training on the full dataset, and a separate classifier that flags likely forgotten rentals.

Use of AI

We used generative AI (Claude, by Anthropic) throughout Stage 2: to propose and implement analyses (the distance-speed decomposition, the feature ablation, the outlier-sensitivity experiment and the day-block bootstrap); to write, run and debug the notebook code; to discuss the problem formulation, including the distinction between a trip's recorded distance and its route; and to draft and edit this report. The design decisions were ours: adopting the revised feature set, comparing three models, and deriving route length from trip history rather than station coordinates. We checked the code, results and text before submission.

6. References

- [1] HSL (Helsinki Region Transport). Helsinki and Espoo city bike trip data. https://hri.fi/data/en_GB/dataset/helsingin-ja-espoon-kaupunkipyorilla-ajatut-matkat
- [2] Finnish Meteorological Institute. Open data, weather observations. <https://en.ilmatieteenlaitos.fi/open-data>
- [3] Jung, A. Machine Learning: The Basics. <https://mlbook.cs.aalto.fi>
- [4] Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1), 73-101.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [6] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

7. Appendix: Code

The complete notebook (ML_bike.ipynb), which preprocesses the data and trains and evaluates all methods, is attached on the following pages.